

Agent Design Document

Trace — Personal Productivity and Task Execution Agent

1. Problem Statement

Managing day-to-day work today means switching between several disconnected tools: a task manager, a notes app, a calendar, and a meeting-notes document, with the user manually copying information between them. Meeting decisions rarely become real tasks unless someone remembers to type them in afterward, and daily planning means manually re-reading a task list and guessing at priority.

Trace solves this by giving the user a single conversational interface backed by a real tool-using AI agent. Instead of navigating forms, the user describes what they want in plain English — creating tasks, searching notes, turning a meeting transcript into action items, or asking for a daily plan — and the agent decides which underlying tool to call, executes it, and reports back. Every action that changes real data is held behind an explicit human approval step, so the agent assists without ever acting fully autonomously.

2. Users

Trace is built for individual knowledge workers managing their own workload — not a multi-user team system. Each browser session is treated as one user's workspace. Two user profiles motivated the design:

- **Busy individual contributors** who attend meetings and need action items turned into real, trackable tasks without manual re-entry.
- **People planning their own day/week** who want an assistant that already knows their current task list and can build a realistic schedule around it, rather than a blank to-do app.

3. Use Cases

The major categories of request Trace is designed to handle, with real examples used during development and testing:

- **Direct questions** — “What's the difference between High and Critical priority?” (answered directly, no tool call)
- **Task management** — “Create a task called Draft report, high priority”, “Show me my high-priority tasks”, “Mark the second one as complete”
- **Notes** — “Save a note about the client call”, “Search my notes for the marketing campaign”
- **Meeting processing** — “Create tasks from these meeting notes: ...”
- **Planning** — “Plan my day, I have 6 hours available”, “Give me my weekly review”

4. Agent Responsibilities

Trace is explicitly allowed to:

- Interpret a natural-language request and decide whether any tool is actually needed.
- Select and call one or more of its registered tools, in sequence, using results from earlier calls to decide the next step.
- Ask a clarifying question instead of guessing when a request is ambiguous.

- Resolve references to something shown earlier in the same session (e.g. “the second one”) using real session memory.
- Propose — but never silently execute — any action that changes stored data or has a real-world effect.
- Record a full, reviewable log of every run it performs.

5. Agent Boundaries

Trace is explicitly not allowed to:

- Execute any write or irreversible action without the user first clicking Approve.
- State a fact, task ID, or note content that did not come from an actual tool result.
- Send a real email — the Send Email tool is deliberately simulated and never opens a real network connection.
- Exceed its configured execution limits: 8 agent steps, 2 retries per tool, and a 30-second tool timeout.
- Repeat the exact same tool call with the exact same arguments twice within one run (loop prevention), or create a second task in one run without triggering approval, even though a single task creation alone does not require it.
- Read or act on another user's session — session memory is scoped strictly to one browser session.
- Expose an API key, a stack trace, or any internal error detail directly to the interface.

6. Tool Catalogue

Every tool the agent can call, its purpose, and whether it requires human approval before executing:

Tool	Purpose	Approval?
create_task	Create one new task	No
create_tasks_bulk	Create several tasks at once (e.g. from meeting notes)	Yes
list_tasks	List tasks, filtered by status / priority / tag	No
update_task	Change a task's title, priority, status, due date, or tags	Yes
complete_task	Mark a task Completed	Yes
delete_task	Permanently delete a task	Yes
search_notes	Keyword search across saved notes, with a relevance score	No
save_note	Save a new note	No
extract_meeting_actions	Turn a transcript into a summary, decisions, and action items	No
generate_work_plan	Build an ordered daily schedule from current tasks	No
detect_overdue_tasks	(Bonus) Find tasks past their due date	No
estimate_task_effort	(Bonus) Estimate hours a task will take	No

send_email	Send an email — simulated only, no real email is sent	Yes
create_reminder	Create a time-based reminder	Yes

7. State Model

Per-run state

Each single call to the agent controller produces an AgentResult: the ordered list of steps taken (stage, a short operational description, which tool if any, and its result), the final answer or error, and any pending approval. This lives only for the duration of that HTTP request/response.

Session state (Requirement 11)

A lightweight, in-memory SessionMemory is kept per browser session: the last 20 messages exchanged, the exact list of tasks most recently shown to the user (so “the second one” resolves to a real task ID), and any preference the user has explicitly stated. This is intentionally not persisted to disk — it is scoped to “the current session” as required, and is cleared on server restart.

Persistent state

Tasks, Notes, and Execution Logs are stored in SQLite via SQLAlchemy and survive server restarts. Pydantic schemas validate every read and write at the boundary between the database and the rest of the app.

8. Approval Model

Approval is required before: creating multiple tasks, updating a task, completing a task, deleting a task, sending an email, and creating a reminder. A single task creation does not require approval on its own — but if create_task is called a second time within the same run, approval is forced at the code level regardless of the model's own judgment, closing a specific loophole where multiple tasks could otherwise be created one approval-free call at a time.

When approval is required, the interface shows: the tool name, a short proposed-action title, the exact input arguments, and a dynamically generated “expected effect” sentence describing what that specific call will do (for example, naming the exact task ID and fields being changed, rather than the tool's generic description). The user can Approve, Reject, or Edit the arguments before approving. Rejecting returns control to the user immediately with nothing executed.

9. Error Strategy

- **Invalid tool arguments** are caught by Pydantic validation before a tool ever runs, with a clear message naming the problem field.
- **Tool failures** are retried up to 2 times with a 30-second timeout per attempt, then reported cleanly rather than left hanging.
- **Invalid or unparseable LLM responses** are caught explicitly and reported as a decision error, instead of crashing on bad JSON.
- **Unexpected failures of any kind** are caught by a final safety net in the Flask routes, which always returns a plain, safe message and never a raw stack trace.
- **Logging failures** (e.g. a database write error while saving the execution log) are isolated so they can never destroy an answer the agent already produced successfully.

- **Loop and duplicate-call prevention** stop a confused model from repeating the same action or running past the step budget.

10. Security Considerations

- The Gemini API key is read only from an environment variable via config.py — it is never hard-coded, never logged, and never sent to the browser.
- A separate SECRET_KEY signs Flask's session cookies, distinct from the API key.
- Execution logs and tool logs record operational facts only (tool name, arguments, result, timing) — API keys and the model's private reasoning are never written to any log.
- Session memory is scoped strictly per session_id; one browser session cannot see another's tasks, notes, or conversation history.
- All database access goes through parameterized SQLAlchemy queries in one repository module — no raw string-built SQL exists anywhere in the app.
- Actions that sound risky but are not yet real (Send Email) are explicitly simulated rather than wired to a real external side effect, and are still approval-gated as if they were real.